Google Summer of Code 2026 Proposal

Quantum Kolmogorov Arnold Networks

- **Project Title:** Quantum Kolmogorov Arnold Networks for High Energy Physics Analysis at the LHC
- **Organization:** ml4-sci , qml-hep , CERN
- **Mentors:** Eric Reinhardt , Dinesh Ramakrishan , KC Kong

Project Summary

Quantum computing presents a promising frontier for High Energy Physics (HEP) analysis at the LHC, where the identification of rare physics signals against immense backgrounds demands increasingly sophisticated computational methods. However, current Quantum Machine Learning implementations often lack a principled architectural foundation .

This project addresses that gap by implementing a Quantum Kolmogorov-Arnold Network (QKAN) grounded in the SineKAN architecture (Reinhardt et al.,2024). Classical SineKANs replace fixed node activations with learnable Fourier edge functions, quantum rotation gates naturally compute sinusoidal functions of their inputs, making SineKAN's basis directly realizable as parameterized quantum circuits.

The proposed QKAN will be implemented in PennyLane/Qiskit and benchmarked against a classical SineKAN reference on a HEP dataset, with the goal of demonstrating that a structurally motivated quantum architecture can meaningfully contribute to LHC physics analysis.

Personal Information

- **Full Name:** Thirukumaran Kathirvelan
- **Email:** thirukumaran3057@gmail.com
- **GitHub:** https://github.com/kthiru-44
- **Linkedin:** https://www.linkedin.com/in/kthiru44/
- **University:** PES University
- **Year of Study**: 2025-2029, Junior
- **Country Residence:** India
- **TimeZone:** IST Asia/Kolkata | UTC+5:30

- **Test Task Solution :** https://github.com/kthiru-44/qmlhep26-tasks.git

---

Qualifications

- IBM Qiskit Advocate: Recognized student contributor with real quantum hardware experience on IBM Quantum Cloud.
- Ranked top 5 at PES University (GPA:9.83) with formal coursework in Quantum Computing, Machine Learning from IBM .
- Proficient in PyTorch with active study of deep learning workflows for particle physics and HEP experiments at LHCb.
- Implementation of New Quantum Feature Maps and Quantum Kernel Methods on MNIST using Qiskit
- Quantum theory to implementation : Independently analyzed Bell inequality violation and quantum advantage in the CHSH game at the circuit level.

Technical Details/Understanding

1. **KAN vs. Standard MLP**

The standard MLP places learnable weights on edges and fixed activation functions on nodes, which:

- Locks the network into a single nonlinearity (ReLU, sigmoid) applied uniformly.
- Limits interpretability as weights are scalar multipliers with no functional meaning.
- Cannot adapt the shape of transformation per edge.

KANs move the learnable nonlinearity from nodes to edges, which solves several fundamental problems:

1. Expressiveness: Each edge learns its own transformation function, not just a scalar weight.
2. Interpretability : Edge functions can be visualized and sometimes simplified to known mathematical forms.

2. **SineKAN vs. B-spline KAN**

The original KAN paper uses B-splines to parameterize edge functions, which:

- Requires managing a grid of knot vectors per edge.
- Has limited smoothness at knot boundaries.
- Computationally expensive on high-dimensional inputs like MNIST.

SineKAN replaces B-splines with a Fourier basis which offers several advantages :

1. **Speed**: Pure matrix operations, no grid management.
2. **Smoothness**: Infinitely differentiable everywhere.
3. **Parameter efficiency**: 2*__K__ parameters per edge encode both amplitude and phase at each frequency

3. **<u>SineKAN and Quantum Circuits</u>**

When the input data x_i is encoded as a rotation angle $\theta = k.x_i$, measurement yields sinusoidal functions of x . The data reuploading scheme (Pérez-Salinas et al.) repeats this encoding across circuit layers, generating progressively higher frequency terms. This means:

1. Each reuploading layer corresponds to one KAN frequency component.
2. Trainable interleaved gates learn the Fourier coefficients a's and b's.
3. The full circuit implicitly computes a SineKAN edge function.

This project aims to make this relation rigorous.

Goals

Goal 1: Formalize the SineKAN - Quantum Correspondence

- Prove the equivalence between SineKAN's Fourier edge functions and data reuploading circuits.
- Show that parameterized rotation gates naturally compute the sine/cosine basis of SineKAN.
- Establish this as the theoretical foundation for the QKAN architecture.

Goal 2: Design and Implement the Quantum KAN in PennyLane

- Build the QKAN circuit with per feature qubit registers encoding each input dimension.
- Circuit specification:

We implement the QKAN as a 2 layer data reuploading circuit for a 4D input using one qubit per feature (4 qubits). Each feature x_i is encoded via $R_x(k.x_i)$, followed by trainable $R_z(\theta_i^{(l)})$ rotations that act as learnable Fourier coefficients. Each layer applies encoding and trainable rotations, while summation is realized through CNOT based entanglement, with outputs obtained via Pauli-Z expectation values $< Z_i >$. The circuit scales linearly in qubits (d) and has depth O(3L) ,making it feasible on NISQ hardware such as IBM Quantum devices.

- Implement parameterized rotation gates as learnable edge functions replacing classical sine/cosine weights.
- Introduce entanglement layers only at summation steps, preserving the KAN structure.
- Validate on MNIST against the classical SineKAN baseline.

Goal 3: Train and Benchmark on a HEP Dataset

- Identify and preprocess a suitable HEP dataset for signal background classification.
- Train the QKAN on LHC relevant tasks and evaluate against the classical SineKAN reference.
- Record accuracy, convergence behavior, and computational cost.

Goal 4: Document and Open Source the Implementation

- Contribute clean, reproducible code to the community.
- Write clear documentation and usage examples for HEP researchers.
- Provide analysis of where quantum architecture offers advantage and where it doesn't.

Estimation of Deliverables and Timeline

Phase 1: Formalizing the SineKAN - Quantum Correspondence (June)

- **Week 1 (June): Research and Mathematical Formalism**
  - Review SineKAN paper and data reuploading paper side by side.
  - Identify the exact mathematical conditions under which the two frameworks are equivalent.
- **Week 2 (June): Establishing Architecture Constraints**
  - Show that a data reuploading circuit with L layers and n qubits computes a finite Fourier series equivalent to a SineKAN edge function.
  - Identify how many qubits and circuit layers are needed to match a given SineKAN configuration, and define boundaries (e.g., multivariate inputs, measurement collapse).

Phase 2: Design, Implement, and Validate Quantum KAN (June & July)

- **Week 3&4 (June): Circuit Architecture Design**
  - Design the QKAN circuit in PennyLane with per-feature qubit registers encoding each input dimension separately.
  - Implement parameterized rotation gates as learnable edge functions replacing classical sine/cosine weights.
- **Week 5 (July): Implementation and Toy Validation**
  - Implement the full QKAN and validate on toy functions to ensure structural identity.
  - Introduce entanglement layers at summation steps.
- **Week 6 (July): MNIST Validation and Debugging**
  - Benchmark against classical SineKAN on MNIST to confirm the quantum implementation is structurally correct.
  - Debug and optimize circuit depth and parameter initialization to mitigate Barren Plateaus.

Phase 3: Train and Benchmark on HEP Datasets (July)

- **Week 7 (July): Data Setup and Preprocessing**
  - Preprocess the HIGGS dataset (Baldi et al., 2014), extracting 28 kinematic features.
  - Handle feature space scaling and class imbalances; prepare 100k event subset for efficient simulation.
- **Week 8 (August): HEP Benchmarking**
  - Train QKAN on signal/background classification.
  - Record primary metrics (AUC-ROC) and secondary metrics (trainable parameters per layer, convergence epochs) against the classical SineKAN reference.

Phase 4: Document and Open Source the Implementation (August)

- **Week 9 (August): Documentation & Code Polish**
  - Write clean, reproducible PennyLane notebooks with inline explanations.
  - Document architectural decisions, limitations, and future directions.
  - Final code submission and documentation handover to the QMLHEP community.

Note :

- Remaining time will be dedicated to testing edge cases, implementing minor ideas discovered during the project, and assisting others in the HEP quantum ML community to build upon the architecture.
- Phase 1 and 2 (correspondence analysis + MNIST validated QKAN implementation) are the primary committed deliverables. Phase 3 (HEP benchmark on HIGGS) is a stretch goal pursued if simulation budget and timeline allow to ensure quality over breadth.

Success Metrics and Validation

- Formal proof of SineKAN with parameterized quantum circuit correspondence documented with clearly stated conditions and limitations.
- Working QKAN implementation in PennyLane preserving KAN structural identity.
- QKAN validated against classical SineKAN baseline on MNIST.
- Quantitative HEP benchmark on accuracy, convergence, and parameter efficiency.
- Testable advantage hypothesis: a QKAN with L reuploading layers will match the AUC-ROC of a classical SineKAN with K = 2L frequency terms using approximately 40% fewer trainable parameters, because Rx gates natively produce both sin and cos basis components simultaneously. Null hypothesis: no statistically significant AUC-ROC difference vs. SineKAN at matched parameter count.

Risks and Mitigations

- **Barren Plateaus:** Vanishing gradients in deep quantum circuits aim to be mitigated by shallow circuit depth and careful parameter initialization.
- **Multivariate Correspondence:** The sin/cos equivalence is clean for single variable functions but breaks down with entanglement , limitations will be documented clearly. Mitigation strategy: for multivariate inputs, treat each feature as an independent qubit register (product encoding). The per feature sin/cos correspondence holds exactly within each register; cross feature interactions are introduced only at the KAN summation node via CNOT, keeping entanglement local and controlled. The full multivariate Fourier decomposition with arbitrary entanglement remains an open research question; this project deliberately operates within the regime where the correspondence is clean and well defined.
- **HEP Data Complexity:** Unfamiliar physics data format may slow preprocessing , dedicated time allocated before any training begins.

Personal Development Goals

- Move from quantum ML as a subject I study to quantum ML as a field I actively contribute to .
- Learn to navigate the gap between theoretical elegance and what actually runs on near term quantum hardware.
- Contribute code that a brilliant community can build on.
- Learn about what goes into making a research grade project and gain firsthand experience implementing ideas at the boundary of what is currently known.

Why I'm the Right Candidate

- Independently connected SineKAN Fourier Basis to Quantum Rotation Gates with Data reuploading circuits . This forms the core of the proposed QKAN Architecture .
- As an IBM Qiskit Advocate, I have hands on experience running quantum circuits on real IBM Quantum Cloud hardware giving me practical understanding of near term device constraints like noise, gate fidelity, and circuit depth limits .
- Implementing Quantum Feature Maps and Quantum Kernel Methods from scratch using Qiskit shows I already have the practical quantum ML foundation this project builds directly on.
- This project sits exactly at the overlap of everything I've been independently pursuing over the past year quantum physics , ML architectures, and Quantum Computing .

I am committed to advancing quantum machine learning methods for LHC physics analysis and contributing meaningfully to the QMLHEP community. I look forward to the opportunity to work on this project during GSoC 2026.

Thank you for considering my proposal !